```
$data = [
    'cats' => ['Jess'],
    'humans' => ['Anna', 'Paul']
];

print http_build_query($data) . PHP_EOL;

Output:
cats%5B0%5D=Jess&humans%5B0%5D=Anna&humans%5B1%5D=Paul
```

The output above might look a little strange; this is because PHP has encoded the query string using a scheme known as "percent encoding" or "url encoding" (you'll hopefully recognize this name from the `application/x-www-form-urlencoded` media type we encountered earlier in the chapter). This is absolutely fine; it's just a safety mechanism, which we'll get to in a moment. The unencoded data looks like this:

```
// $data = [...
print urldecode(http_build_query($data));

Output:
cats[0]=Jess&humans[0]=Anna&humans[1]=Paul
```

The use of explicit numbered array indices is just how PHP rolls; `"cats[0]=Jess&humans[0]=Anna&humans[1]=Paul"` and `"cats[]=Jess&humans[]=Anna&humans[]=Paul"` will result in exactly the same structured data when parsed.

So, what was all that `%5B %5D` stuff about? Chances are that you've encountered these weird-looking sequences in URLs before. URLs (or more precisely, URIs) require that certain reserved characters be encoded into sequences starting with a `%` character. These are characters with a "reserved purpose" in the URI specification, for example, the `&` character separates key/value pairs in a query string. The problem is this: what if the values of the query parameters contain reserved characters? Just writing them into a query string verbatim will screw up the syntax and create havoc with the resulting parsed data. Let's start with the obvious: the `&` character delimits key/value pairs in the query, so if the values themselves contain that character, we'd need to deal with it somehow:

```
$data = [
    'var1' => '&val1',
    'var2' => '&val2',
];
```